

RED TEAM LEARNING PATH

MODULE 09

Vulnerability Assessment

Omarchy / Arch Linux Edition

*From component evidence to validated remediation and rescan.***CORE RULE**

Scanner output adalah candidate evidence, bukan otomatis finding final. Presence, applicability, exploitability, exposure, dan business context harus dipisahkan supaya report tidak overclaim.

Version 1.0 | 04 Oktober 2026

Field	Value
Prerequisite	Module 00–08 — scope/RoE, recon, networking, service enumeration, evidence handling, scripting dasar
Core tools	Trivy, jq, Python, optional Syft (SBOM), browser untuk CVE/NVD/FIRST/CISA
Lab safety	Hanya menganalisis manifest dependency lokal yang sengaja outdated — dependency tidak di-install/dieksekusi
Outcome	Mengubah scanner hit jadi keputusan triage yang bisa dipertanggungjawabkan, lengkap dengan remediation + rescan evidence

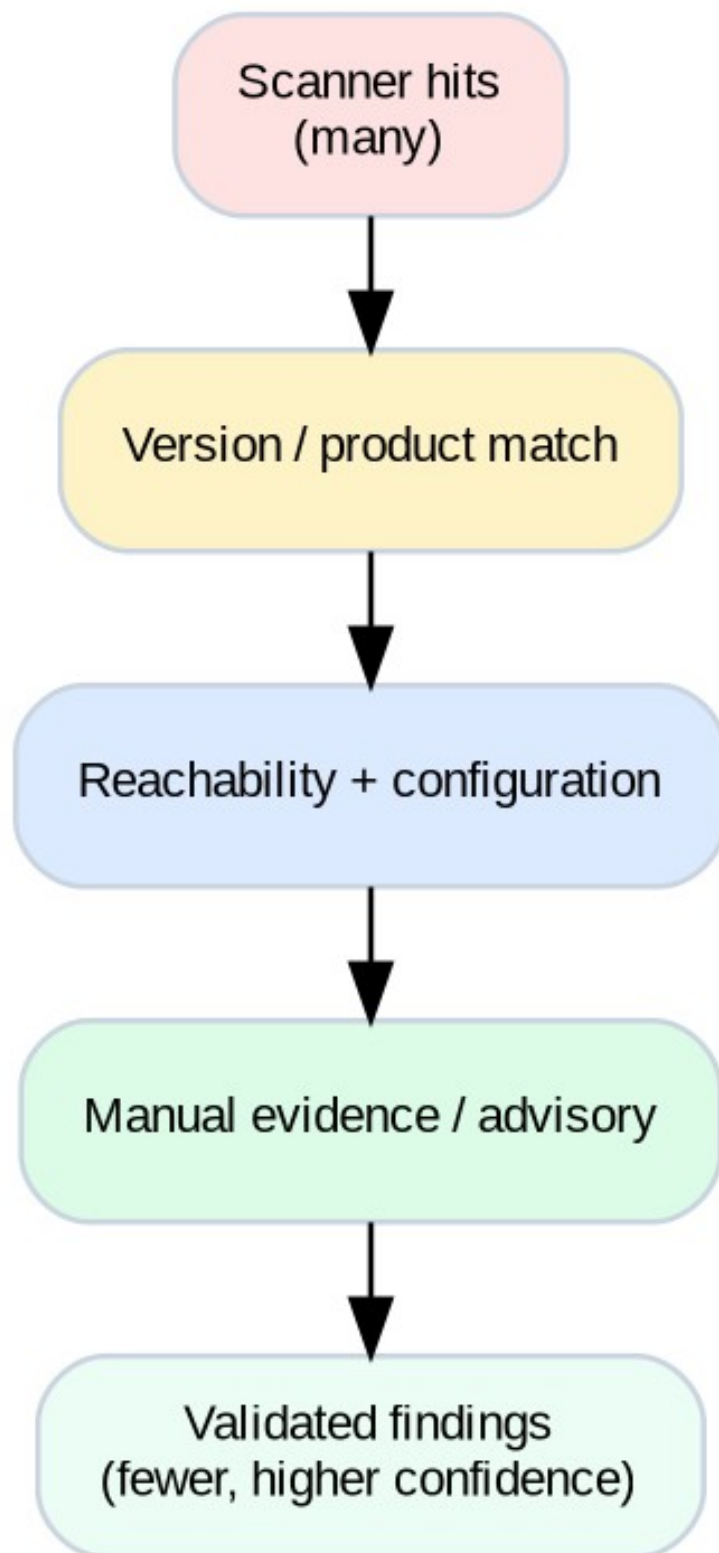
```
omarchy pkg add trivy syft jq python curl
```

Trivy tersedia di repo resmi Arch dan bisa mendeteksi known vulnerability pada OS packages, language packages, container image, dan sebagian software component. Greenbone/OpenVAS dibahas sebagai contoh infrastructure vulnerability-management platform, tapi bukan dependency wajib untuk lab ini.

1. Learning Objectives

- [] Membedakan **weakness**, **vulnerability**, **CVE**, **CWE**, **CPE/package identity**, **severity**, dan **risk** — enam istilah yang sering dipakai bergantian padahal beda makna.
- [] Membaca hasil vulnerability scanner tanpa langsung percaya label severity-nya.
- [] Menggunakan **CVSS v4.0** sebagai severity framework — bukan sebagai risk score tunggal.
- [] Menilai product/version match, fixed version, reachability, konfigurasi, dan evidence sebelum menyatakan "affected".
- [] Memahami false positive, false negative, duplicate, accepted risk, dan not-applicable.
- [] Memprioritaskan remediation memakai technical severity + exploitation evidence + exposure + business criticality.
- [] Melakukan rescan untuk **membuktikan** remediation — bukan sekadar menutup ticket.

2. Mental Model: Detection → Validation → Decision



A large candidate set should shrink as evidence quality increases.

Vulnerability assessment bukan lomba menghasilkan angka finding sebanyak mungkin. Goal-nya adalah membuat daftar **keputusan yang bisa dipertanggungjawabkan**: apa yang benar-benar affected, seberapa serius, kenapa prioritasnya begitu, dan apa bukti remediation-nya.

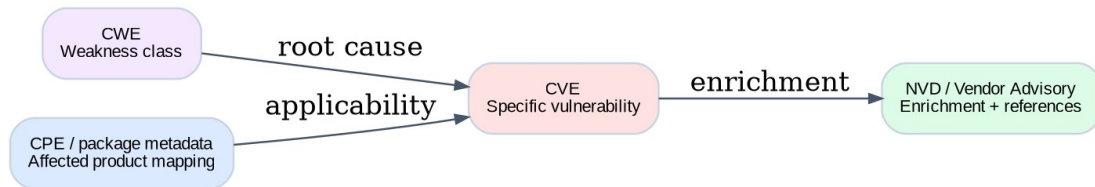
```

Candidate set (scanner raw output, besar & noisy)
  ↓ validasi product/version
  ↓ validasi reachability/config
  ↓ validasi exploitation evidence (KEV, advisory)
  ↓ tambahkan business context
Validated finding set (lebih kecil, defensible)

```

Semakin tinggi kualitas evidence yang lo kumpulkan, semakin kecil (dan semakin akurat) daftar finding yang tersisa.

3. Terminologi Inti: CWE, CVE, CPE, NVD, KEV



How common vulnerability identifiers and enrichment relate conceptually.

Istilah	Makna praktis
CWE	Kelas weakness/root cause (mis. <i>improper input validation</i>). Bukan instance di product tertentu.
CVE	Identifier untuk vulnerability yang <i>publicly disclosed</i> pada product/version tertentu.
CPE / package identity	Cara mengaitkan finding ke product/platform/package yang affected — mapping-nya bisa imperfect.
NVD	Database NIST yang meng- <i>enrich</i> CVE dengan metadata: CVSS, CWE, references, applicability.
Vendor advisory	Sumber penting untuk affected/fixed version, prasyarat konfigurasi, workaround, patch.
KEV	Katalog CISA untuk vulnerability yang diketahui sudah dieksploitasi di dunia nyata.
Weakness ≠ vulnerability. MITRE CWE mendefinisikan weakness sebagai kondisi yang <i>dapat berkontribusi</i> pada vulnerability. CVE adalah instance vulnerability pada product tertentu. Jangan menulis report seolah CWE-79 dan sebuah CVE adalah jenis identifier yang sama level-nya.	

4. Candidate Finding vs Validated Finding

Pertanyaan	Evidence yang dicari
Product benar?	Package manager, SBOM, banner + manual verification, vendor inventory
Version affected?	Vendor advisory / CVE record / distro advisory — perhatikan backport patch
Component reachable?	Network path, API/function exposure, feature enabled, access control
Config prerequisite terpenuhi?	Feature flag, plugin/module, auth requirement, deployment mode

Fix tersedia?	Fixed version, patch, workaround, compensating control
Eksplorasi aktif?	CISA KEV / vendor threat intel / public exploitation evidence
Business impact?	Asset criticality, data sensitivity, privilege, blast radius, availability requirement

Backport trap.

Distro/vendor Linux bisa meng-*backport* security patch **tanpa mengubah version string** upstream seperti yang lo harapkan. Version string saja tidak selalu cukup untuk menyatakan vulnerable — selalu cek advisory vendor/distro sebelum final decision.

5. CVSS v4.0: Severity, Bukan Risk Tunggal

CVSS v4.0 adalah framework FIRST untuk mengomunikasikan karakteristik teknis dan severity. Ada empat metric group:

Metric group	Pertanyaan
Base	Seberapa berat karakteristik teknis intrinsik vulnerability-nya?
Threat	Apa yang berubah berdasarkan exploit maturity/aktivitas threat saat ini?
Environmental	Bagaimana karakteristik environment kita mengubah dampak/importance-nya?
Supplemental	Info tambahan (safety, automatable, recovery, provider urgency) — tidak mengubah score final

NVD reminder.

NVD secara eksplisit menyatakan **CVSS adalah ukuran severity, bukan ukuran risk**. Jangan menerjemahkan CVSS 9.8 menjadi "risk 9.8" tanpa business/exposure context.

Saat mencatat CVSS, selalu simpan **score DAN vector**:

CVSS-B: 8.7
Vector: CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/...

Vector membuat orang lain bisa memahami *metric choices* di baliknya — bukan sekadar melihat angka akhir. Gunakan **FIRST calculator** untuk scoring; jangan menebak vector dari label scanner.

6. Scanner Mechanics & Limitations

- Signature/database scanner bekerja dari evidence: package name/version, CPE, advisory metadata, configuration check, atau network probe.
- Database freshness** memengaruhi hasil — scanner hari ini dan bulan depan bisa memberi hasil berbeda pada input yang sama persis.
- Unauthenticated network scan cuma melihat apa yang exposed dari luar; credentialed/local scan biasanya punya visibility jauh lebih tinggi.
- Scanner bisa salah mapping package/product, gagal mengenali vendor backport, atau tidak tahu apakah vulnerable code path benar-benar *reachable*.
- False negative** juga mungkin: komponen custom, third-party binary, runtime-only dependency, shadow IT, atau database coverage yang tidak lengkap.

Jenis scanner

Jenis	Contoh	Kekuatan	Keterbatasan
SCA / filesystem	Trivy, Dependency-Check	Dependency/component	Presence ≠ exploitability

		visibility	
Container/image	Trivy	OS + language packages sebelum deploy	Image state ≠ runtime exposure
Infrastructure VM	Greenbone/OpenVAS	Network services + vulnerability tests	Butuh tuning, credential, impact awareness
Authenticated host	Scanner agent/credentialed scan	Package/config visibility tinggi	Credential handling & privilege harus aman
DAST/web	Burp/ZAP class tools	Runtime behavior	Coverage bergantung crawl/auth/business flow

7. Prioritization: Severity + Threat + Exposure + Business

Prioritas yang berguna menjawab "apa yang harus diperbaiki dulu?" — bukan sekadar mengurutkan angka CVSS.

Signal	Contoh interpretasi
Known exploited	CISA KEV / vendor konfirmasi active exploitation → urgency naik
Internet exposed	Attack surface terbuka tanpa internal foothold → urgency naik
Privileged service	Compromise memberi dampak SYSTEM/root/domain → urgency naik
Business critical	Downtime/data loss berdampak tinggi → impact context naik
Strong compensating control	Segmentation, feature disabled, WAF/ACL, vulnerable path tidak reachable → residual risk bisa turun
No fix yet	Pakai vendor workaround/mitigation + monitoring — jangan sekadar "accept" tanpa owner/expiry

KEV adalah priority signal

, bukan auto-critical. CISA KEV sengaja dibuat untuk membantu organisasi memprioritaskan vulnerability yang sudah diketahui dieksploitasi — tapi tetap validasi apakah product/version lo affected dan apakah asset-nya relevan dengan environment yang di-KEV-kan.

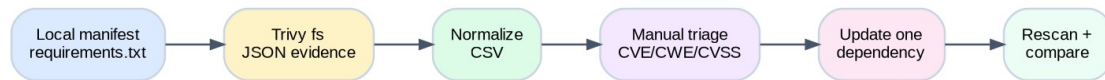
8. Remediation Quality

- Tentukan root cause/component yang harus diubah.
- Pilih patch/upgrade/vendor workaround yang **supported**.
- Uji compatibility di environment yang sesuai.
- Deploy dengan change control.
- **Rescan/re-test** menggunakan evidence path yang sama.
- Bandingkan before/after dan simpan buktinya.
- Tutup finding **hanya** kalau acceptance criteria terpenuhi.

Fix ≠ silence scanner.

Menyembunyikan finding via ignore/suppression bukan remediation. Suppression hanya legitimate kalau ada *documented justification*: false positive, not-applicable, accepted risk dengan owner/expiry, atau pernyataan VEX/vendor yang valid.

9. Lab Design



Module 09 lab scans a local manifest only; vulnerable dependencies are never executed.

Fixture berisi beberapa dependency versi lama untuk menghasilkan candidate finding. **Exact CVE count bisa berubah** karena database vulnerability terus diperbarui — itu justru bagian dari pelajaran: evidence harus punya timestamp dan database context yang jelas, bukan angka yang dianggap tetap selamanya.

```

module09-vb/
├── 00-scope/
├── 01-target/requirements.txt      ← fixture: dependency sengaja outdated, TIDAK di-install
├── 02-scans/trivy.json            ← raw evidence
├── 03-normalized/findings.csv     ← view terstruktur untuk triage
├── 04-triage/decisions.csv        ← decision log manual
├── 05-remediation/
├── 06-evidence/sha256-manifest.txt
└── 07-summary/vulnerability-summary.md
  
```

`run-trivy.sh` hanya mau men-scan path yang punya marker `.module09-training-target` — guardrail ini sengaja mencegah script latihan dipakai scan filesystem arbitrary secara tidak sengaja.

10. Hands-on Lab 1 — Workspace & Training Marker

```

cd modules/09-vulnerability-assessment/labs
./va-init.sh module09-vb
cat module09-vb/00-scope/README.md
  
```

11. Hands-on Lab 2 — Create Intentionally Outdated Manifest

```

./make-fixture.sh module09-vb
cat module09-vb/01-target/requirements.txt
  
```

Isinya lima package versi lama (`urllib3`, `Jinja2`, `PyYAML`, `Django`, `Pillow`) — murni sebagai *scanner input*, bukan sesuatu yang di-install atau dieksekusi.

Safe fixture.

Kita cuma memberi scanner sebuah manifest untuk dipetakan terhadap known vulnerability data — tidak ada dependency vulnerable yang benar-benar jalan di mesin lo.

12. Hands-on Lab 3 — First Trivy Scan

```

./run-trivy.sh module09-vb

jq '.Results[] | {Target,Type,count:(.Vulnerabilities // [] | length)}' \
  module09-vb/02-scans/trivy.json
  
```

First run biasanya perlu mengunduh vulnerability database dulu. **Simpan waktu scan dan tool version** karena output bisa berubah seiring update database:

```

trivy --version
date -u +%FT%TZ
  
```

13. Hands-on Lab 4 — Normalize Evidence

```
python normalize-trivy.py \
  module09-va/02-scans/trivy.json \
  --output module09-va/03-normalized/findings.csv

column -s, -t < module09-va/03-normalized/findings.csv | less -S
```

CSV-nya berisi target, type, package, installed, fixed, vulnerability, severity, status, primary_url. **JSON tetap jadi raw evidence** — CSV cuma view terstruktur untuk triage. Jangan hapus sumber mentah setelah normalisasi.

14. Hands-on Lab 5 — Manual CVE Validation

Pilih satu CVE dari CSV. Untuk CVE tersebut, jawab secara tertulis:

- [] Apa product/package yang affected?
- [] Version range affected apa?
- [] Apa fixed version menurut scanner/vendor advisory?
- [] CWE/root cause apa yang terkait?
- [] Apa prerequisite/attack vector-nya?
- [] Apakah hanya *presence* yang terbukti, atau *code path/reachability* juga terbukti?
- [] Apakah ada distro advisory spesifik yang mengubah applicability?

Gunakan CVE Record/vendor advisory sebagai identitas vulnerability, NVD sebagai enrichment/reference, dan CWE untuk weakness class. **Jangan andalkan satu blog/exploit repo acak** sebagai source of truth.

15. Hands-on Lab 6 — Baca CVSS v4, Jangan Copy Blindly

- Buka FIRST CVSS v4 calculator.
- Ambil CVSS vector dari advisory authoritative kalau tersedia.
- Baca tiap metric dan tulis kenapa nilainya masuk akal.
- Bandingkan Base dengan Threat/Environmental context di lab lo.
- Catat score + vector — bukan score saja.

Exercise.

Ambil satu finding severity HIGH dari scanner. Bayangkan asset-nya cuma offline build artifact dan vulnerable dependency itu **tidak pernah dieksekusi**. Technical severity CVE-nya tidak berubah, tapi remediation priority di environment itu bisa berubah drastis. Jelaskan perbedaannya di notes lo.

16. Hands-on Lab 7 — Triage Decision Log

```
python finding-note.py \
  module09-va/04-triage/decisions.csv \
  --cve CVE-YYYY-NNNN \
  --package urllib3 \
  --decision needs-more-evidence \
  --reason "package present; runtime reachability not established" \
  --priority P3
```

--decision cuma menerima tiga nilai: validated, not-applicable, needs-more-evidence; --priority menerima P1–P4 atau defer. Decision log ini memaksa lo memisahkan "scanner says vulnerable" dari "gue sudah punya evidence untuk claim ini".

17. Hands-on Lab 8 — Optional SBOM View

```
syft dir:module09-va/01-target \
  -o cyclonedx-json=module09-va/06-evidence/sbom.cdx.json
```

SBOM membantu menjawab *komponen apa yang terdeteksi*. **SBOM bukan vulnerability report** dan tidak membuktikan exploitability — ia jadi input inventory yang bisa dikorelasikan dengan vulnerability intelligence di langkah berikutnya.

18. Hands-on Lab 9 — Remediate One Dependency

Dari finding yang punya FixedVersion, pilih satu package. **Jangan copy version dari modul ini** — fixed version bisa berubah; selalu pakai data scanner/vendor yang current.

```
python remediate-one.py \
  module09-va/01-target/requirements.txt \
  --package urllib3 \
  --version <FIXED_VERSION_FROM_CURRENT_EVIDENCE>
```

Script ini membuat backup requirements.txt.bak secara otomatis. Review diff-nya sebelum rescans:

```
diff -u module09-va/01-target/requirements.txt.bak \
  module09-va/01-target/requirements.txt || true
```

19. Hands-on Lab 10 — Rescan & Compare

```
cp module09-va/02-scans/trivy.json \
  module09-va/05-remediation/trivy-before.json

./run-trivy.sh module09-va

cp module09-va/02-scans/trivy.json \
  module09-va/05-remediation/trivy-after.json
```

Normalize before/after secara terpisah, lalu cek apakah CVE yang ditargetkan hilang/berubah. Finding lain boleh tetap ada — goal lab ini adalah membuktikan **satu** remediation dengan evidence yang repeatable, bukan membersihkan seluruh daftar.

20. Evidence Integrity

```
./evidence-manifest.sh module09-va
cat module09-va/06-evidence/sha256-manifest.txt
```

Integrity scope.

SHA-256 membuktikan file **tidak berubah** sejak manifest dibuat. Hash **tidak** membuktikan scanner-nya benar, target-nya benar, atau interpretasi lo benar — itu tetap tanggung jawab analisis manual.

21. Mini Project — Vulnerability Case File

```
module09-va/
├── 00-scope/
├── 01-target/requirements.txt
├── 02-scans/trivy.json
├── 03-normalized/findings.csv
├── 04-triage/decisions.csv
├── 05-remediation/
├── 06-evidence/
└── 07-summary/vulnerability-summary.md
```

Summary minimal mencantumkan: target/component, tool + DB timestamp, candidate finding, manual validation status, severity/vector, asumsi exposure/business, remediation, hasil rescans, dan remaining uncertainty.

22. Practical Challenge — 45 Menit

- [] Buat workspace dan fixture baru.
- [] Jalankan Trivy dan simpan raw JSON.
- [] Normalize ke CSV.
- [] Pilih minimal dua CVE dari package berbeda.
- [] Untuk masing-masing: cek authoritative CVE/advisory, affected range, fixed version, CWE (kalau ada), dan CVSS vector.
- [] Tulis satu finding validated/likely dan satu finding needs-more-evidence/not-applicable lengkap dengan alasan.
- [] Pilih satu dependency dengan fixed version, lakukan remediation manifest.
- [] Rescans dan buktikan before/after.

[] Buat SHA-256 evidence manifest.

[] Tulis summary tanpa melebih-lebihkan exploitability.

Challenge rule.

Lo tidak perlu menjalankan exploit untuk memvalidasi vulnerability assessment dasar. Untuk banyak finding, evidence product/version/configuration/advisory saja sudah cukup. Exploitation baru dilakukan kalau scope, safety, dan objective engagement memang membutuhkan proof yang lebih dalam.

23. Common Mistakes

Kesalahan	Impact	Perbaikan
CVSS = risk	Prioritas salah	Gabungkan severity dengan threat/exposure/business context
Version match = vulnerable	False positive	Cek vendor advisory, backport, config, reachability
Ignore fixed version	Finding tidak actionable	Selalu capture remediation source/fixed version
Scan DB stale	Miss/incorrect finding	Catat DB/tool timestamp, update sesuai policy
Unauth scan dianggap lengkap	False sense of coverage	Pahami visibility; pakai authenticated/local evidence bila diizinkan
Jalankan exploit untuk semua CVE	Risiko operasional	Gunakan validasi least-invasive dulu
Tutup ticket setelah patch deploy	Belum terbukti	Rescan/retest, simpan evidence before/after
Suppress tanpa rationale	Risiko tersembunyi	Owner, reason, expiry, compensating control harus jelas

24. Knowledge Check

- Apa beda CWE dan CVE?
- Kenapa NVD menyatakan CVSS bukan risk score?
- Apa beda presence, applicability, reachability, dan exploitability?
- Kenapa vendor backport bisa membuat version-only matching salah?
- Apa beda authenticated dan unauthenticated vulnerability scan?
- Kapan sebuah finding bisa dianggap not-applicable?
- Kenapa KEV memengaruhi prioritization?
- Apa fungsi fixed version dalam remediation workflow?
- Kenapa raw JSON scanner tetap disimpan setelah CSV dibuat?
- Apa bukti minimum untuk menutup finding setelah patch?

25. Exit Criteria

[] Bisa membedakan CWE, CVE, CVSS, NVD, vendor advisory, dan KEV.

[] Tidak menyebut scanner hit sebagai confirmed vulnerability tanpa validasi.

[] Bisa membaca Trivy JSON dan membuat normalized inventory.

- [] Bisa mencatat score + CVSS vector dan menjelaskan kenapa severity bukan risk tunggal.
- [] Bisa membuat triage decision dengan evidence/rationale.
- [] Bisa memperbaiki satu dependency berdasarkan current fixed-version evidence lalu rescans.
- [] Bisa menjelaskan false positive dan false negative secara realistis.
- [] Memahami kapan exploitation **tidak** diperlukan untuk vulnerability assessment.

26. Deliverables

```
09-vulnerability-assessment/
├── README.md
├── docx/Red-Team-Module-09-Vulnerability-Assessment-0marchy.docx
├── pdf/Red-Team-Module-09-Vulnerability-Assessment-0marchy.pdf
├── labs/
│   ├── va-init.sh           - scaffold workspace + training marker
│   ├── make-fixture.sh      - bikin manifest dependency sengaja outdated (tidak dieksekusi)
│   ├── run-trivy.sh         - jalankan Trivy, hanya pada path bermarker training
│   ├── normalize-trivy.py   - Trivy JSON → CSV findings
│   ├── finding-note.py     - append baris ke triage decision log (CSV)
│   ├── remediate-one.py    - update satu pin version di requirements.txt + backup
│   └── evidence-manifest.sh - SHA-256 manifest untuk scans/normalized/triage
```

27. Cheat Sheet

```
# Install tools
omarchy pkg add trivy syft jq python curl

# Workspace + safe fixture
./labs/va-init.sh module09-va
./labs/make-fixture.sh module09-va

# Scan
./labs/run-trivy.sh module09-va
jq '.Results[] | {Target,Type,count:(.Vulnerabilities // [] | length)}' \
  module09-va/02-scans/trivy.json

# Normalize
python labs/normalize-trivy.py module09-va/02-scans/trivy.json \
  --output module09-va/03-normalized/findings.csv

# Manual triage log
python labs/finding-note.py module09-va/04-triage/decisions.csv \
  --cve CVE-YYYY-NNNN --package PACKAGE \
  --decision needs-more-evidence --reason "..." --priority P3

# Evidence integrity
./labs/evidence-manifest.sh module09-va
```

28. References

- NIST NVD — Vulnerability Detail Pages — <https://nvd.nist.gov/vuln/Vulnerability-Detail-Pages>
- FIRST — CVSS v4.0 User Guide — <https://www.first.org/cvss/v4.0/user-guide>
- MITRE CWE — FAQ — <https://cwe.mitre.org/about/faq>
- CVE Program — Glossary — <https://www.cve.org/ResourcesSupport/Glossary>
- CISA — Known Exploited Vulnerabilities Catalog — <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
- Trivy — Vulnerability Scanning — <https://trivy.dev/docs/latest/scanner/vulnerability/>
- Arch Linux — trivy package — https://archlinux.org/packages/extra/x86_64/trivy/
- OWASP Dependency-Check — <https://owasp.org/projects/dependency-check>
- Greenbone Community Documentation — <https://greenbone.github.io/docs/latest/>
- NIST SP 800-115 — Technical Guide to Information Security Testing — <https://csrc.nist.gov/pubs/sp/800/115/final>

Next module:

10-burp-suite-fundamentals — proxy, HTTP history, Repeater, scope, TLS interception, request editing, evidence discipline, dan local web lab workflow.